Architecture

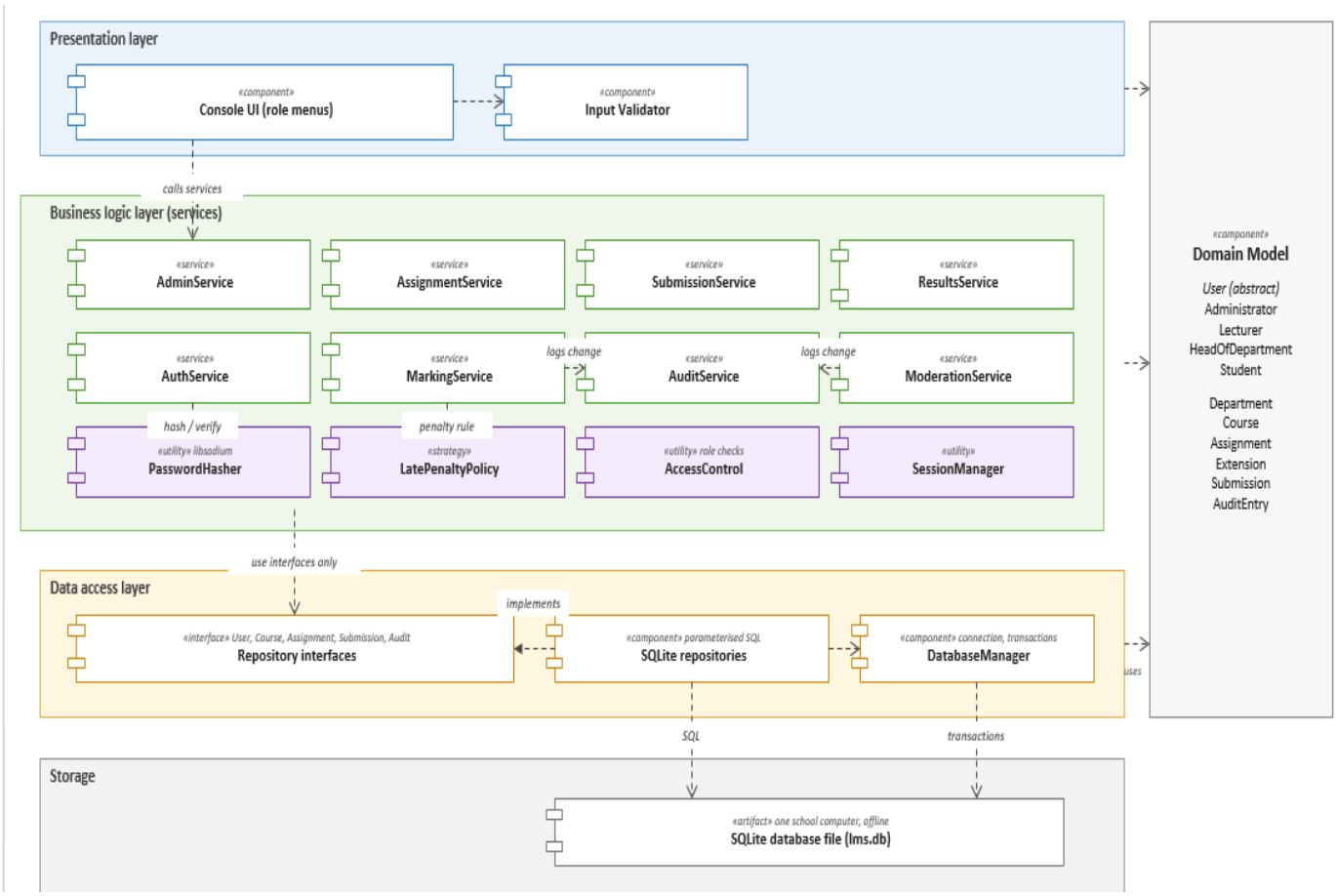
CampusNode: Moderated Learning Management System

1. Architectural style

The system uses a **layered architecture**. It has four layers and each layer only dependent on the layer below it:

1. **Presentation:** the console menus. They show information and collect input.
2. **Business logic:** services that apply the school rules (late penalty, extensions, moderation, audit log).
3. **Data access:** repositories. This is the only code that writes SQL.
4. **Storage:** the SQLite database file.

The **domain model** (User, Course, Assignment, Submission and so on) is shared by all layers. The business layer calls repository *interfaces*, not SQLite directly, so it does not depend on the database.



Each layer depends on the layer below it and business layer depends on repository interfaces, never on SQLite. Dashed arrows show dependencies. Every service calls Access Control before acting (role check, NFR5).

2. Design Rationale

Each part of the system has one job: show menus, apply school rules, or store data. A layered design keeps these apart. This makes the business rules easy to test without a database, keeps all SQL in one place, and is simple enough for one person to build in two weeks.

Alternatives Rejected

1. Model-View-Controller (MVC). MVC splits a program into a model (the data and rules), a view (what the user sees) and a controller (which turns user actions into changes to the model). It works well for graphical and web programs, where several views of the same data must stay in step. For example, changing a mark might have to refresh a table, a chart and a summary at the same time. Our system is a console program that shows one screen at a time, so there are no linked views to keep in step, and MVC gives little benefit. It also has a practical risk. In small programs the controller tends to collect business rules such as the late penalty and the moderation rules, because it is the first place that receives the user's request. Those rules are the most important and most heavily tested part of the system. They need to sit in a separate layer that has no console or database code, so I can test them with automated tests and reach 70% coverage. I kept the useful idea from MVC, which is that the interface should be thin: the console menus only collect input and call services. If a graphical interface is added later, MVC would become a better choice, and the services could be reused unchanged.

2. Client-server with a database server (for example MySQL or PostgreSQL). In this design the data lives in a server program, and many computers connect to it over the network. It is the usual choice when many people use a system at the same time. It was rejected for three reasons. First, we decided that the system runs on one school computer and works offline (A6 and BR8), so there is nobody to connect from another machine. Second, a database server must be installed, secured, updated and backed up on old school computers by a technician who is not a database expert, and the system would also need networking code and network security. That adds cost and risk (risks R6 and R9) for no benefit at this size. Third, SQLite already handles 800 students easily and needs no server. If the school later wants many computers to use the system, only the data access layer would need to change, because the services use repository interfaces and never SQLite directly.